

Yellowstone/Geyser Observability Extension for the Solana Containerised Testbed v0.2.0

Oleksandr Khoshaba
Vinnitsia National Technical University, Ukraine
Oleksandr.Khoshaba@gmail.com
ORCID: [0000-0001-5375-6280](https://orcid.org/0000-0001-5375-6280)

Version 0.2.0

Technical Note DOI: [10.5281/zenodo.TODO](https://doi.org/10.5281/zenodo.TODO)
Software v0.2.0 DOI: [10.5281/zenodo.20132465](https://doi.org/10.5281/zenodo.20132465)
Original Technical Report DOI: [10.5281/zenodo.20098291](https://doi.org/10.5281/zenodo.20098291)

Abstract

This technical note documents the Yellowstone/Geyser observability extension introduced in Solana Containerised Testbed v0.2.0. The extension adds a Yellowstone/Geyser-enabled Solana v1.18.25 local validator image, a containerised Solana latency monitor, and a Prometheus-compatible metrics endpoint. The resulting observability core exposes Solana JSON-RPC on 127.0.0.1:8899, Yellowstone/Geyser gRPC on 127.0.0.1:10000, and Prometheus metrics on 127.0.0.1:9464. The implementation preserves the no-AVX2 compatibility objective established in the earlier validator containerisation work and was validated on legacy x86_64 hardware. This note describes the motivation, architecture, container images, release workflow, validation procedure, limitations, and research roadmap for using the v0.2.0 observability core as the baseline for Dataset layer 0, Kubernetes deployment, controlled load response experiments, and later adaptive control research.

Keywords: Solana; Yellowstone; Geyser; Prometheus; observability; testbed; containerisation; Podman; Docker Compose; no-AVX2; legacy x86_64; dataset generation; blockchain performance.

Contents

1 Introduction	2
2 Baseline: Solana Containerised Testbed v0.1.x	2
3 Motivation for the v0.2.0 Observability Extension	3
4 v0.2.0 Architecture	3
5 Yellowstone/Geyser Integration	4
6 Containerised Latency Monitor	4
7 Prometheus Metrics Endpoint	5
8 Docker Hub Images and Immutable Digests	5
9 Release Workflow	5

10 no-AVX2 Validation	6
11 Dataset Layer 0	6
12 Research Roadmap	7
13 Limitations	8
14 Conclusion	8

1 Introduction

The Solana Containerised Testbed project provides a reproducible local Solana testbed for dataset generation, performance observation, and future adaptive transaction-load control experiments. The first technical report documented the initial containerisation of a Solana v1.18.25 local validator and wallet bootstrap workflow for legacy x86_64 hardware without AVX2 support [3]. That earlier milestone established the basic validator and wallet-init components and demonstrated that a source-built no-AVX2 Solana image could run on older hardware where standard prebuilt binaries failed.

The present technical note documents the next milestone, Solana Containerised Testbed v0.2.0. This version adds a Yellowstone/Geyser observability layer and a containerised Solana latency monitor. The main purpose of this extension is to turn the testbed from a runnable local validator environment into an observable research environment.

The v0.2.0 release is not intended to be a throughput benchmark by itself. Instead, it establishes an observability core:

```
validator + Yellowstone/Geyser + monitor + Prometheus metrics
```

This observability core becomes the basis for Dataset layer 0, whose purpose is to validate that metrics can be collected reproducibly before introducing synthetic load generation, dashboards, MPC controllers, reinforcement learning agents, or Agave-based deployments.

2 Baseline: Solana Containerised Testbed v0.1.x

The v0.1.x baseline provided a containerised reproduction of the original virtual-machine workflow. It separated the system into a validator service and a wallet bootstrap service.

The original validator workflow was based on:

```
solana-test-validator \
  --ledger ~/solana-localnet/ledger \
  --reset \
  --bind-address 127.0.0.1 \
  --rpc-port 8899
```

The wallet bootstrap workflow used the Solana command-line interface to configure the RPC endpoint, inspect the payer balance, request an airdrop, and verify the resulting balance:

```
PAYER=6avCzMrjUDebRYtSoQ6GPQENjoxDaD2Udik8JzRnKbtb
solana config set --url http://127.0.0.1:8899
solana balance "$PAYER"
solana airdrop 10000 "$PAYER"
solana balance "$PAYER"
```

A major engineering issue in the v0.1.x stage was CPU compatibility. The target legacy host used an Intel Core i7-3667U CPU with AVX support but without AVX2 support. Standard Solana prebuilt binaries and the official Solana container image were unsuitable on this host. The project therefore built Solana v1.18.25 from source with AVX2 disabled and packaged the resulting binaries into compatibility images.

The v0.1.x release established:

- a no-AVX2 Solana validator image;
- a wallet-init image;
- Docker Hub publication of runtime images;
- release-mode Compose workflows;

- Zenodo software archiving and DOI assignment;
- a technical report documenting the initial containerisation work.

3 Motivation for the v0.2.0 Observability Extension

The next research requirement was not to immediately introduce external load generation or adaptive control. Before a controlled load response dataset can be trusted, the underlying observation path must be validated.

The v0.2.0 extension therefore focuses on the following question:

Can the local Solana testbed expose reproducible runtime observations through Yellowstone/Geyser and Prometheus-compatible metrics on legacy x86_64 hardware?

This question must be answered before the project moves toward:

- controlled synthetic load generation;
- knee-point probing;
- MPC-based transaction-load control;
- single-agent reinforcement learning;
- multi-agent reinforcement learning;
- Agave-based research branches;
- Kubernetes-based deployment of the full experimental stack.

The v0.2.0 milestone therefore introduces an observability core but deliberately avoids mixing it with the later control and learning layers.

4 v0.2.0 Architecture

The v0.2.0 architecture contains three primary runtime services:

1. **Validator:** a Solana v1.18.25 local validator image with Yellowstone/Geyser enabled.
2. **Wallet-init:** a one-shot container that funds the predefined payer account.
3. **Monitor:** a containerised Solana latency monitor that connects to Yellowstone/Geyser and exposes Prometheus metrics.

The release-mode architecture is summarised as follows:

Host		
127.0.0.1:8899	-> validator:8899	Solana JSON-RPC
127.0.0.1:10000	-> validator:10000	Yellowstone/Geyser gRPC
127.0.0.1:9464	-> monitor:9464	Prometheus metrics
Compose network		
wallet-init	-> validator:8899	
monitor	-> validator:10000	

The service-level responsibilities are intentionally separated. The validator runs the local Solana chain, the wallet-init container performs deterministic local funding, and the monitor observes the validator through Yellowstone/Geyser and publishes metrics.

5 Yellowstone/Geyser Integration

The v0.2.0 validator image includes the Yellowstone/Geyser plugin library:

```
/usr/local/lib/libyellowstone_grpc_geyser.so
```

The validator is configured with a Geyser plugin configuration file:

```
/etc/solana/yellowstone-geyser.json
```

The relevant runtime environment values are:

```
ENABLE_GEYSER=true  
GEYSER_CONFIG=/etc/solana/yellowstone-geyser.json  
RPC_PORT=8899
```

Inside the validator container, the Yellowstone/Geyser gRPC service listens on:

```
0.0.0.0:10000
```

The host binds this endpoint to:

```
127.0.0.1:10000
```

The Geyser-enabled validator image was built from a no-AVX2-compatible Solana baseline image and a source-built Yellowstone/Geyser plugin. The no-AVX2 build policy is an important part of the release because the project targets legacy x86_64 systems in addition to newer AVX2-capable hosts.

6 Containerised Latency Monitor

The monitor component is a Go-based service imported from the existing Solana latency research workflow and adapted for the containerised testbed.

The monitor connects to the validator through the Compose service name:

```
validator:10000
```

The container-oriented monitor configuration uses:

```
rpc: http://validator:8899  
grpc: validator:10000  
interval: 5s  
metrics:  
  prometheus_port: 9464
```

The monitor exposes Prometheus-compatible metrics on:

```
0.0.0.0:9464
```

The host maps this to:

```
127.0.0.1:9464
```

The monitor was normalised to use English source comments, configuration text, and Prometheus HELP strings. This was done to make the component suitable for public research software documentation and later publication.

7 Prometheus Metrics Endpoint

The v0.2.0 monitor exposes Prometheus-compatible metrics at:

```
http://127.0.0.1:9464/metrics
```

A typical check is:

```
curl -s http://127.0.0.1:9464/metrics | head -n 40
```

Expected metric families include:

```
solana_slot_interval_seconds
solana_transaction_latency_seconds
solana_subscription_errors_total
```

Example HELP text after normalisation:

```
# HELP solana_slot_interval_seconds Time interval between consecutive Solana slots,
    used to measure slot production stability.
# HELP solana_transaction_latency_seconds Latency distribution from transaction
    broadcast to confirmation.
# HELP solana_subscription_errors_total Total number of errors observed in the
    subscription stream.
```

These metrics form the initial basis for Dataset layer 0. Dataset layer 0 is intended to validate observability, not to characterise maximum throughput.

8 Docker Hub Images and Immutable Digests

The v0.2.0 release uses the following Docker Hub images:

```
docker.io/khoshaba/solana-localnet-validator:v1.18.25-noavx2-ivybridge-yellowstone
docker.io/khoshaba/solana-wallet-init:v1.18.25-noavx2-ivybridge
docker.io/khoshaba/solana-latency-monitor:v0.2.0
```

The immutable image digests are recorded in the repository file:

```
release/v0.2.0-image-digests.txt
```

This approach separates the human-readable image tags from the reproducibility-critical image digests. The Zenodo software release archives the source code, Dockerfiles, Compose files, documentation, and digest metadata, while Docker Hub remains the operational registry for pulling and running the images.

9 Release Workflow

The v0.2.0 release workflow is defined by:

```
compose.yellowstone.release.yaml
```

The release workflow is started with:

```
podman compose -f compose.yellowstone.release.yaml up
```

The validator health check is performed through Solana JSON-RPC:

```
curl -s http://127.0.0.1:8899 \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc": "2.0", "id": 1, "method": "getHealth"}'
```

The expected response is:

```
{"jsonrpc": "2.0", "result": "ok", "id": 1}
```

The payer balance is checked inside the validator container:

```
podman exec -it solana-localnet-validator \
  solana --url http://127.0.0.1:8899 \
  balance 6avCzMrjUDebRYtSoQ6GPQENjoxDaD2Udik8JzRnKbtb
```

The expected balance after wallet bootstrap is:

```
10000 SOL
```

The Yellowstone/Geyser port can be checked inside the validator container:

```
podman exec -it solana-localnet-validator \
  ss -ltnp | grep 10000
```

The monitor metrics endpoint is checked with:

```
curl -s http://127.0.0.1:9464/metrics | head -n 40
```

10 no-AVX2 Validation

A central constraint of the project is support for legacy x86_64 hosts without AVX2 support. The v0.2.0 release preserves this constraint by using a no-AVX2 Solana baseline and by building the Yellowstone/Geyser plugin with conservative CPU compatibility settings.

The relevant target host class includes:

```
Architecture: x86_64
AVX: yes
AVX2: no
Example CPU: Intel Core i7-3667U
```

The Yellowstone/Geyser-enabled validator image and monitor image were tested on a no-AVX2 host. The validation criteria included:

- the validator container does not crash with illegal-instruction failures;
- Solana JSON-RPC health returns `ok`;
- the payer balance is available after wallet bootstrap;
- the Yellowstone/Geyser gRPC port listens on port 10000;
- the monitor container starts successfully;
- Prometheus metrics are available on `127.0.0.1:9464`;
- metric HELP strings and source text are normalised to English.

11 Dataset Layer 0

The next immediate research step is Dataset layer 0: an observability validation dataset. The purpose of this dataset is to prove that the observability core is reproducible before introducing external load generation.

A candidate output layout is:

```
output/datasets/layer0-observability/<run_id>/
  metadata.json
  samples.csv
  samples.jsonl
  raw-metrics.prom
```

Candidate fields include:

```
timestamp_utc
run_id
host_id
cpu_model
avx2_present
git_commit
testbed_version
validator_image
monitor_image
validator_health
payer_balance_sol
metrics_scrape_ok
slot_interval_p50
slot_interval_p90
slot_interval_p99
slot_interval_sum
slot_interval_count
transaction_latency_p50
transaction_latency_p95
transaction_latency_p99
transaction_latency_sum
transaction_latency_count
subscription_errors_total
```

The initial recommended run profile is:

```
duration: 60 seconds
sample interval: 2 seconds
```

Dataset layer 0 is deliberately not a controlled load response dataset. It is a validation artifact for the observation pipeline.

12 Research Roadmap

The v0.2.0 release anchors the first research level: the observability core. The accepted roadmap is layered as follows:

1. Observability Core.
2. Dataset layer 0: observability validation.
3. Kubernetes deployment of the observability core.
4. Dataset layer 1: controlled load response.
5. MPC adaptive control.
6. Single-agent reinforcement learning.
7. Multi-agent reinforcement learning.

8. Agave research branch.

A key methodological decision is that Kubernetes is treated as an environment and deployment layer, not as the learning agent. The Kubernetes stage should initially deploy the observability core only: validator, wallet-init job, monitor, services, and persistent storage. It should be future-controller-ready but should not include MPC, RL, MARL, load generation, dashboarding, or Agave in the first step.

Another key decision is to avoid moving directly to MARL. The intended control progression is:

controlled load response -> system model -> MPC -> single-agent RL -> MARL
--

This preserves interpretability and creates baselines for later learning-based methods.

13 Limitations

The v0.2.0 release has several limitations:

- It validates observability but does not yet provide a controlled load response dataset.
- It does not yet containerise the existing load generator and dashboard workflow.
- It does not yet include Kubernetes manifests.
- It does not yet include MPC, single-agent RL, or MARL.
- It does not yet include an Agave validator branch.
- Prometheus summary metrics are useful for validation, but later datasets may require lower-level event traces or additional raw measurements.
- The no-AVX2 compatibility claim should remain tied to the tested host class and should be expanded only after broader hardware validation.

14 Conclusion

Solana Containerised Testbed v0.2.0 extends the earlier validator and wallet-init containerisation work with a Yellowstone/Geyser observability layer and a containerised latency monitor. The release exposes Solana JSON-RPC, Yellowstone/Geyser gRPC, and Prometheus-compatible metrics through a reproducible Compose workflow based on Docker Hub images. The system was validated on legacy x86_64 hardware without AVX2 support and is intended to serve as the baseline for Dataset layer 0, Kubernetes deployment of the observability core, controlled load response experiments, and later adaptive control research.

The main contribution of v0.2.0 is therefore not maximum throughput measurement, but the creation of a reproducible and observable local Solana testbed suitable for systematic dataset generation and future control-oriented experimentation.

Related Identifiers

- Technical Note DOI: [10.5281/zenodo.TODO](https://doi.org/10.5281/zenodo.TODO).
- Software concept DOI: [10.5281/zenodo.20095383](https://doi.org/10.5281/zenodo.20095383).
- Software v0.2.0 DOI: [10.5281/zenodo.20132465](https://doi.org/10.5281/zenodo.20132465).
- Original technical report DOI: [10.5281/zenodo.20098291](https://doi.org/10.5281/zenodo.20098291).
- GitHub repository: <https://github.com/okhoshaba/solana-containerised-testbed>.

References

- [1] Khoshaba, *Solana Containerised Testbed v0.2.0*, Zenodo software release. DOI: [10.5281/zenodo.20132465](https://doi.org/10.5281/zenodo.20132465).
- [2] Khoshaba, *Solana Containerised Testbed*, Zenodo software concept record. DOI: [10.5281/zenodo.20095383](https://doi.org/10.5281/zenodo.20095383).
- [3] Khoshaba, *Containerising a Solana v1.18.25 Local Testbed for Reproducible Dataset Generation on Legacy x86_64 CPUs*, Zenodo technical report. DOI: [10.5281/zenodo.20098291](https://doi.org/10.5281/zenodo.20098291).
- [4] Khoshaba, *Solana Containerised Testbed GitHub Repository*. <https://github.com/okhoshaba/solana-containerised-testbed>.
- [5] Khoshaba, *Docker Hub Image: solana-localnet-validator*. <https://hub.docker.com/r/khoshaba/solana-localnet-validator>.
- [6] Khoshaba, *Docker Hub Image: solana-latency-monitor*. <https://hub.docker.com/r/khoshaba/solana-latency-monitor>.